Performance of Bitonic MergeSort: A Comparative Study

Abstract

The growing demand for efficient data processing has driven the development of various parallel sorting algorithms. This paper examines and compares the performance of Bitonic Merge Sort with other widely used parallel algorithms such as parallel bucket sort, Quick Sort and Merge Sort, focusing on execution time and hardware efficiency. Bitonic Merge Sort is noted for its structured, deterministic behavior, making it well-suited for parallel platforms like GPUs. In contrast, algorithms like Parallel Merge Sort and Quick Sort offer greater flexibility but are less predictable, often better suited for CPU-based systems. While Parallel Bucket Sort offers fast performance but is memory intensive. The study emphasizes that selecting an appropriate sorting algorithm should be guided by both the characteristics of the data and the underlying hardware architecture.

Keywords: Parallel sorting algorithm, Power efficiency, Time complexity

1. Introduction

There are two categories of sorting networks: parallel and pipeline. The sorted outputs in pipeline networks are generated one after the other after a predetermined amount of time. On the other hand, a parallel sorting network, which completes all comparisons at once and provides the final sorted result in a single step (Jelodari et al., 2020).

Among various parallel sorting algorithms, the Bitonic MergeSort Network-Based method stands out due to its structured and predictable nature. It sorts data step-by-step through comparisons and swaps, making it well-suited for mapping onto parallel hardware. It operates effectively on fixed interconnection networks, FPGAs, and GPUs (DINA BITTON et al., 1984). In GPU computing, Bitonic Mergesort (BM) is a widely used algorithm, well-suited for parallel architectures. It features minimal inter-process communication, supports in-place implementation, and fits the SIMD (Single Instruction, Multiple Data) model efficiently (Abulnaja et al., 2018). In terms of power efficiency, Bitonic MergeSort consumes less power and utilizes the GPU more efficiently than Advanced QuickSort when executing the same workload (Abulnaja et al., 2018; Ikram et al., 2022).

In contrast, algorithms like QuickSort and MergeSort (Shared Memory) also support parallelism but rely on recursion and dynamic memory access. These are more flexible and deliver excellent performance on CPUs with multiple cores and thread-level parallelism (DINA BITTON et al., 1984). While, for large datasets that cannot be fully loaded into memory, the file-based parallel multiway mergesort is a better choice. It reduces disk I/O operations, making it ideal for databases and external sorting tasks (DINA BITTON et al., 1984).

From such point view each sorting algorithm has its own cons and pros depending on the nature of data. Therefore, this study is important to know when to use the most common parallel sorting algorithm depending on their performance chracteratics.

2. Bitonic Merge Sort: Detailed Analysis

Bitonic Merge Sort is a parallel sorting algorithm that takes advantage of bitonic sequences, special sequences that first increase and then decrease, or vice versa (Joella Lobo & Sonia Kuwelkar, 2020). The algorithm operates in two primary steps. First, it transforms the input array (of size $n=2^k$) into a bitonic sequence by dividing it into two halves: one sorted in ascending order and the other in descending order. This forms a bitonic sequence, which is a prerequisite for the second step. In the **bitonic merge** step, the sequence is recursively split and compare and exchange operations are performed between elements at positions i and i+n/2. This comparison ensures that all values in the first half are less than those in the second half, producing two smaller bitonic sequences. The process is applied recursively until the entire sequence becomes sorted. The algorithm proceeds in k phases, where each phase builds larger bitonic sequences from smaller ones) (Rocha & Silva, 2015).

In **Phase 1**, adjacent pairs of values are converted into bitonic sequences by sorting one value ascending and the other descending.

Phases 2 to k−1 involve sorting each m-bit bitonic sequence and merging adjacent sequences into 2m-bit bitonic sequences, continuing until a full n-bit bitonic sequence is obtained.

Phase k completes the sorting by merging this final bitonic sequence into a fully ordered list. Each phase i requires i parallel steps, and the total number of steps is the sum of the first k integers, which equals k(k+1)/2. As k = log n, the total parallel time complexity becomes $O((\log n)^2)$, making Bitonic Merge Sort highly efficient for parallel computing environments

(Rocha & Silva, 2015). Here is an example in figure 1 and figure 2, how to work bitonic merge sort with an input array of 3, 5, 8, 9, 7, 4, 2, 1.

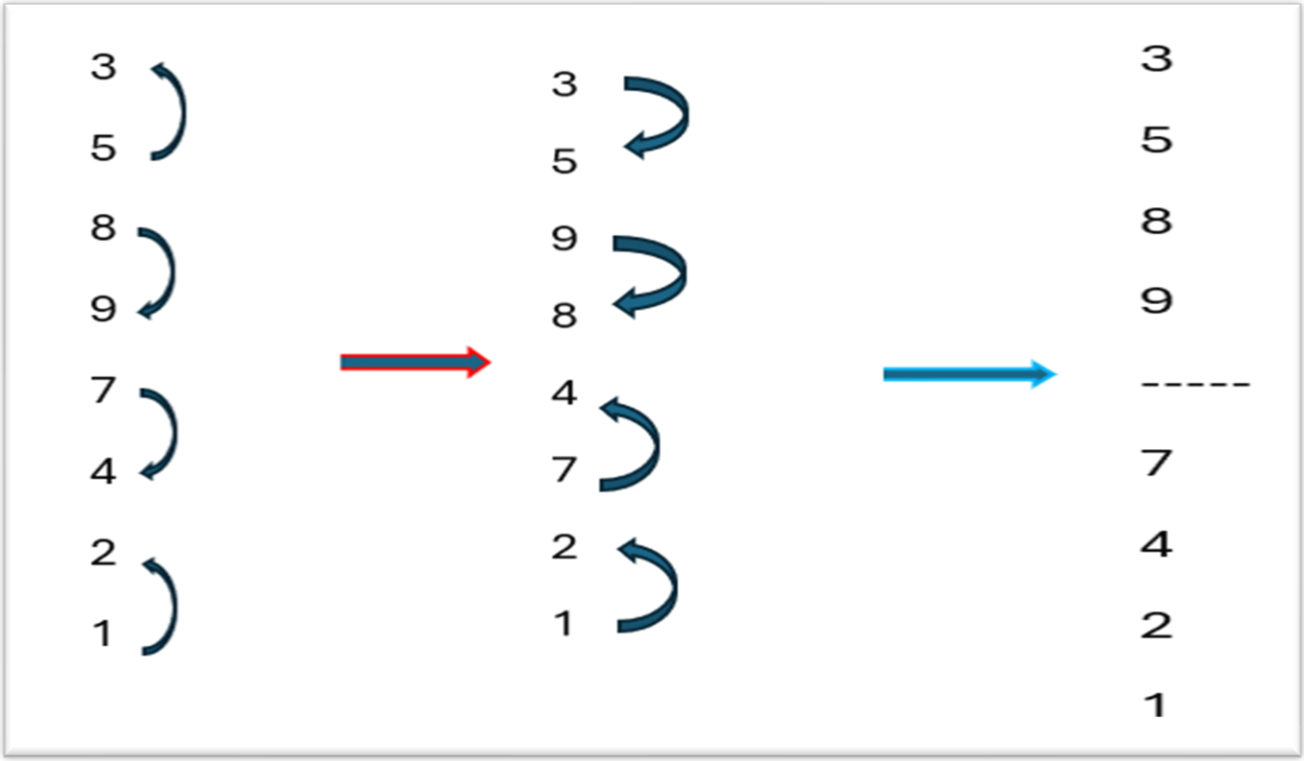

Figure 1 steps of bitonic sequence (GeeksforGeeks, 2023)

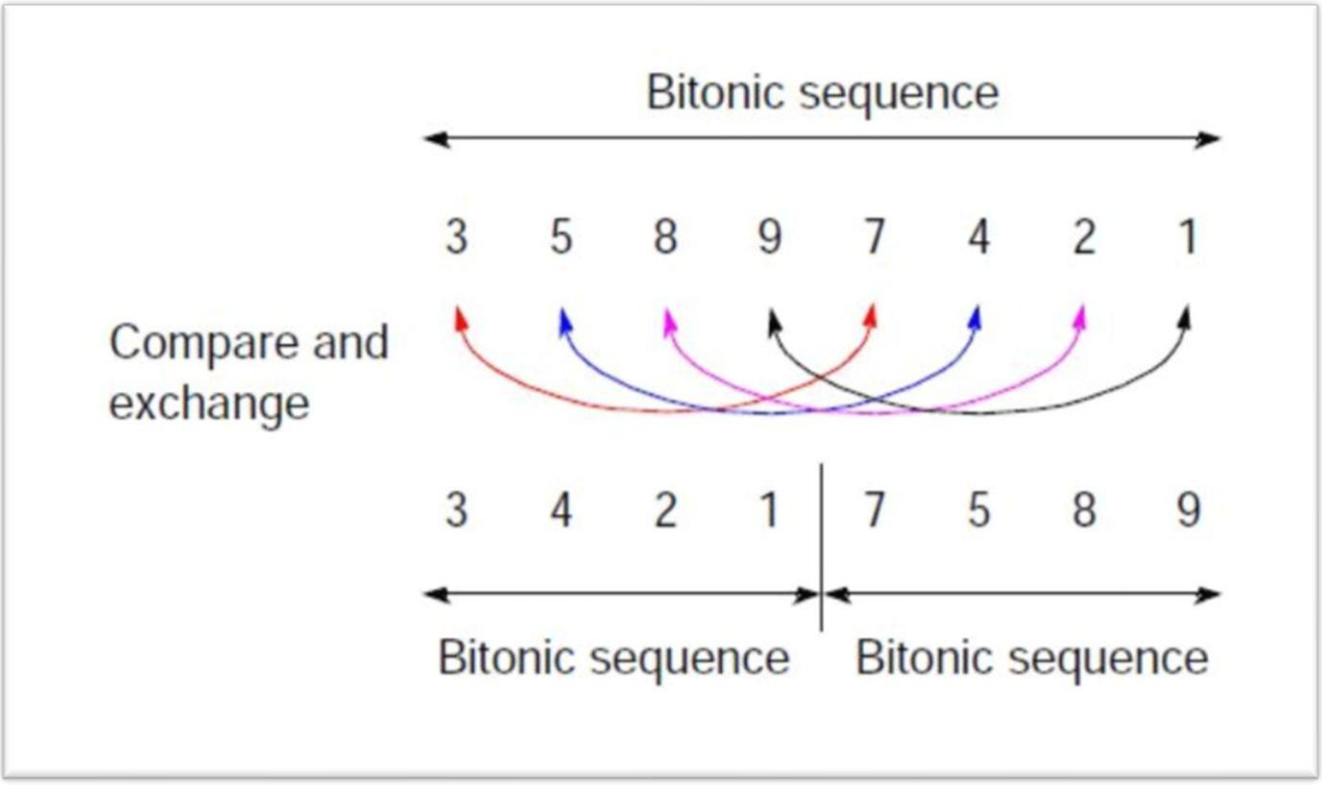

Figure 2 steps of bitonic sequence (Rocha & Silva, 2015)

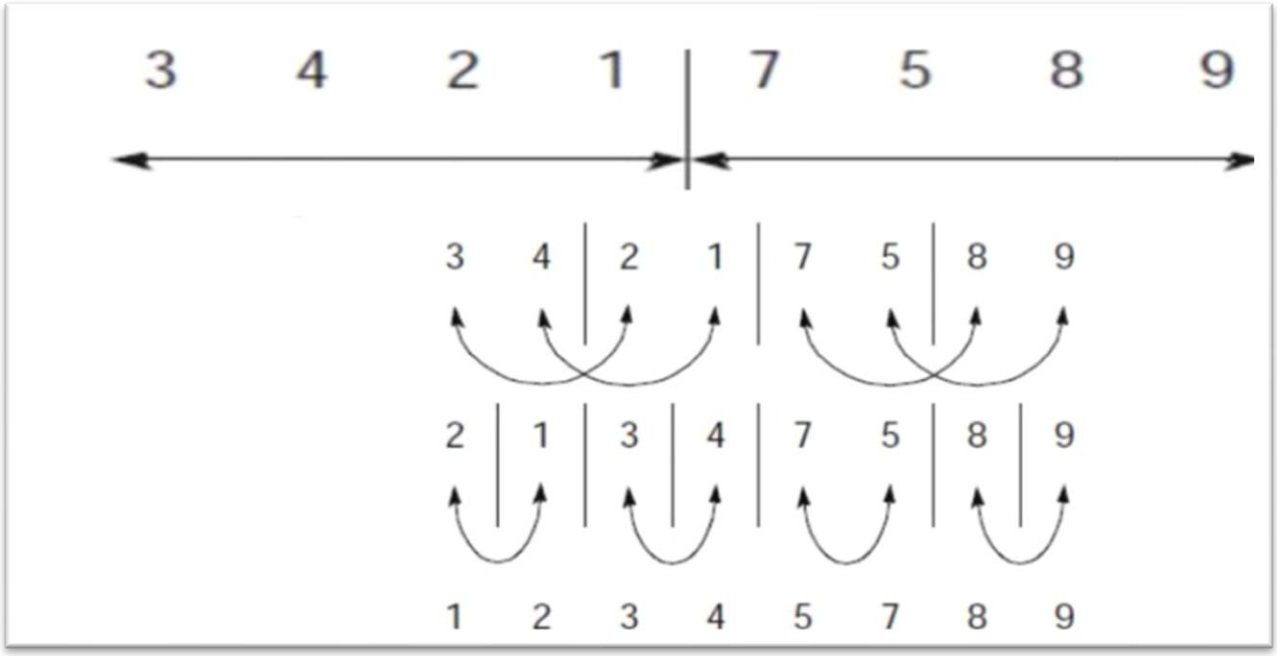

Figure 3 bitonic merge sort (Rocha & Silva, 2015)

3. Data Analysis

The performance of the sorting algorithms was analyzed under various hardware configurations, including GPUs and CPUs, with a focus on execution time, memory usage, and power efficiency. This data analysis helps to draw conclusions about the suitability of each algorithm based on different workload scenarios.

4. Performance of common parallel sorting algorithms

Sorting performance is influenced primarily by two factors: the nature of the algorithm and the underlying hardware architecture (Jan et al., 2012). Some studies showed that parallel bucket sort algorithm achieves O (log n) time using n processors, offering fast, scalable performance with good load balancing. However, it requires more than O (n log n) space, making it memory-intensive (Hirschberg, 1978).

In bitonic merge sort due to ease of parallelization, achieves up to 2300x speed-up over the odd-even merge sorting technique when executed on a Quadro 6000 GPU (Jan et al., 2012) . Although the Odd-Even Merge Sort and Bitonic Merge Sort share similar time (O $(\log n)^2$) and space complexities (O(n)), Bitonic Merge Sort is generally more efficient in terms of speedup and scalability (Joella Lobo & Sonia Kuwelkar, 2020). It is better suited for parallel processing, with more predictable memory access patterns, while Odd-Even Merge Sort requires more temporary memory and control overhead (Joella Lobo & Sonia Kuwelkar, 2020). The other type of sorting algorithm, parallel Quick Sort performs in O ($\log^2 n$) parallel time using O(n) additional space for partition. However, unlike merge sort, it's harder to parallelize efficiently due to issues like unbalanced partitions and irregular data access (Wikipedia, 2025). Both Bitonic Merge Sort and Parallel Merge Sort achieve O ($\log^2 n$) parallel time, but Bitonic Merge Sort is preferred on GPUs for its regular structure, while Parallel Merge Sort suits CPU-based parallel processing due to its work efficiency (Wikipedia, 2024).

5. Algorithm and Implementation

```
Algorithm BitonicMergeSort (A [], low, high, direction):

  if low >= high:

    return
```

```
    mid = (low + high) / 2  // Index-based partitioning
    // Step 1: Sort first half in ascending order
    BitonicMergeSort (A, low, mid, 1)
    // Step 2: Sort second half in descending order
    BitonicMergeSort (A, mid + 1, high, 0)
    // Step 3: Merge into one sorted sequence
    BitonicMerge (A, low, high, direction)


Algorithm BitonicMerge (A [], low, high, direction):
    if high > low:
        mid = (high - low) / 2   //  Number of elements partitioning
        // Step 1: Compare and swap elements
        for i = low to (low + mid - 1):
            if direction == 1:  // Ascending
                if A[i] > A[i + mid]:
                    swap A[i] and A[i + mid]
            else:               // Descending
                if A[i] < A[i + mid]:
                    swap A[i] and A[i + mid]
        // Step 2: Recursively merge both halves
        BitonicMerge (A, low, low + mid - 1, direction)
        BitonicMerge (A, low + mid, high, direction)
```

6. Summary

This paper categorizes sorting networks into pipeline and parallel types and emphasizes the superiority of parallel methods for high-speed execution. Bitonic Merge Sort stands out for its structured, GPU-friendly nature, while Parallel Merge Sort and Quick Sort are more suited to CPU architecture due to their recursive and dynamic memory features. Comparative analysis reveals that Bitonic Merge Sort achieves higher speed-up and better power efficiency in GPU settings, making it ideal for SIMD execution. In contrast, Parallel Merge Sort remains more efficient in terms of total work and memory usage, preferred in CPU-based systems. The paper underscores that while several algorithms offer similar theoretical complexities, practical performance is heavily influenced by hardware compatibility and implementation overhead.

7. Conclusion

Bitonic Merge Sort excels in highly parallel environments, especially on GPUs, due to its deterministic pattern, minimal inter-process communication, and in-place execution. However, for CPU-based or distributed systems requiring lower overall computational work, Parallel Merge Sort is more effective. Despite similar parallel time complexity, real-world efficiency varies depending on workload, memory constraints, and platform characteristics. This study confirms that no single sorting algorithm is universally optimal, each has strengths suited to specific scenarios, reinforcing the need to align algorithm choice with both the nature of the data and the target hardware architecture.

8. Reference

Abulnaja, O. A., Ikram, M. J., Al-Hashimi, M. A., & Saleh, M. E. (2018). Analyzing Power and Energy Efficiency of Bitonic Mergesort Based on Performance Evaluation. *IEEE Access*, *6*. https://doi.org/10.1109/ACCESS.2018.2861571

GeeksforGeeks. (2023, December 28). *Bitonic Sort | GeeksforGeeks*. https://www.geeksforgeeks.org/bitonic-sort/

DINA BITTON, AVID J. DeWITT, AVID K. HSIAO, & ISHANKAR MENON. (1984). A Taxonomy of Parallel Sorting. *Computing Surveys*, *16*(3).

Hirschberg, D. S. (1978). Fast parallel sorting algorithms. *Communications of the ACM*, *21*(8), 657–661. https://doi.org/10.1145/359576.359582

Ikram, M. J., Saleh, M. E., Al-Hashimi, M. A., & Abulnaja, O. A. (2022). Investigating the effect of varying block size on power and energy consumption of GPU kernels. *Journal of Supercomputing*, *78*(13). https://doi.org/10.1007/s11227-022-04473-9

Jan, B., Montrucchio, B., Ragusa, C., Khan, F. G., & Khan, O. (2012). FAST PARALLEL SORTING ALGORITHMS ON GPUS. *International Journal of Distributed and Parallel Systems (IJDPS*, *3*(6). https://doi.org/10.5121/ijdps.2012.3609

Jelodari, P. T., Kordasiabi, M. P., Sheikhaei, S., & Forouzandeh, B. (2020). An O(1) time complexity sorting network for small number of inputs with hardware implementation. *Microprocessors and Microsystems*, *77*. https://doi.org/10.1016/j.micpro.2020.103203

Joella Lobo, & Sonia Kuwelkar. (2020). *Performance Analysis of Merge Sort Algorithms*. https://doi.org/10.1109/ICESC48915.2020.9155623

Rocha, R., & Silva, F. (2015). *Parallel Sorting Algorithms*. https://www.dcc.fc.up.pt/~ricroc/aulas/1516/cp/apontamentos/slides_sorting.pdf

Wikipedia. (2024, November 14). *Merge algorithm - Wikipedia*. https://en.wikipedia.org/wiki/Merge_algorithm?utm_source=chatgpt.com

Wikipedia. (2025, April 19). *Quicksort - Wikipedia*. https://en.wikipedia.org/wiki/Quicksort?utm_source=chatgpt.com